

Automata and Complexity

Aamod Varma

CS - 4510, Spring 2026

Contents

1	Regular Languages	2
1.1	Finite Automata	2
1.1.1	Computation	3
1.1.2	Regular Operations	3
1.2	Nondeterminism	4

Chapter 1

Regular Languages

1.1 Finite Automata

Finite automata is a model for a computer with an extremely limited amount of memory. For instance, consider a controller for an automatic door. We can have two states, open or closed. And 4 input conditions, front, rear, neither and both.

Definition. A *finite automaton* is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where,

1. Q is a finite set called the states
2. Σ is a finite set called the alphabet
3. $\delta : Q \times \Sigma \rightarrow Q$ is the transition function
4. $q_0 \in Q$ is the start state
5. $F \subseteq Q$ is the set of accept states.

Remark. By this definition we can have $\emptyset \subseteq F$ and hence have no accept states.

Note. Note also that δ a function defined from the domain $Q \times \Sigma$ and hence every state has a transition to another state for any given input alphabet.

Consider the following automata M_1 ,

1. $Q = \{q_1, q_2, q_3\}$
2. $\Sigma = \{0, 1\}$
3. δ is,

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

4. q_1 is the start state
5. $F = \{q_2\}$

Now let A be the set of all strings that our machine accepts, then we say that A is the language of machine M and $L(M) = A$.

In this case if we have,

$$A = \{w \mid w \text{ contains at least one 1 and an even number of 0s follow the last 1} \}$$

Then we have M recognizes A .

Note. Note that even number of zeros also means zero zeroes and hence we can have the string end with just 1.

1.1.1 Computation

Let M be a finite automata (with the usual 5-tuple representation). Let $w = w_1w_2 \dots w_n$ be a string where each w_i is a member of Σ . We say M accepts w if there exists a sequence of states $r_0, \dots, r_n$ in Q such that,

1. $r_0 = q_0$
2. $\delta(r_i, w_{i+1}) = r_{i+1}$, for $i = 0, \dots, n-1$
3. $r_n \in F$

Remark. Condition 1 says that we start in the start state. Condition 2 says that we transition according to our string and Condition 3 states that we end with a accept state.

Definition. A language is called a *regular language* if a finite automata recognizes it.

1.1.2 Regular Operations

Definition. Let A, B be languages. We define the regular operations **union**, **concatenation** and **star** as follows,

1. Union: $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$
2. $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$
3. $A^* = \{x_1 \dots x_k \mid k \geq 0 \text{ and each } x_i \in A\}$

Example. If Σ is the regular alphabet and $A = \{good, bad\}$ and $B = \{boy, girl\}$ then,

- $A \cup B = \{good, bad, boy, girl\}$
- $A \circ B = \{goodboy, goodgirl, badboy, badgirl\}$
- $A^* = \{\varepsilon, good, bad, goodgood, goodbad, badgood, badbad, goodgoodgood, goodgoodbad, goodbadgood, goodbadbad, \dots\}$

◇

Theorem 1.1. The class of regular languages is closed under the union operation.

Note. This means that if A_1 and A_2 are regular languages then so is $A_1 \cup A_2$

Proof. If A_1 is regular there exists a finite automaton M_1 that accepts it. Similarly we have M_2 . Now the goal is to construct a new finite automaton M that can accept both languages. We can do this by having M simulate both M_1 and M_2 . Define $M = \{Q, \Sigma, \delta, q_0, F\}$ as follows,

1. $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ and } r_2 \in Q_2\}$
2. Σ is the same as both M_1 and M_2 (the union of both their alphabets if M_1, M_2 differ in their alphabet)
3. δ is defined from $(Q_1 \times Q_2) \times \Sigma \rightarrow (Q_1 \times Q_2)$ as follows for $(r_1, r_2) \in Q$ we have,

$$\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$$

4. q_0 is the pair (q_1, q_2)
5. $F = \{(r_i, r_j) \mid r_i \in F_1 \text{ or } r_j \in F_2\}$

□

Theorem 1.2. Regular languages are closed under concatenation

Proof. Consider we have two regular languages A_1, A_2 corresponding to M_1 and M_2 . We need a new finite automata M that is able to accept the concatenation of languages from A_1 and A_2 . So essentially from every accept state of M_1 once the string from A_1 is finishes we need a mechanism to switch to M_2 in order to compute the string from A_2 . This means that from the accept states we might have multiple arrows for the same alphabet.

To allow this, we need to introduce a new concept called **nondeterminism**.

□

1.2 Nondeterminism

When a machine is in a given state and reads the next input symbol, if the next state is predetermined (i.e only a single choice is there) then we called it a deterministic computation. In a nondeterministic machine, several choices may exists for the next state at any point.

This is a generalisation of a deterministic machine (so every determinstic finite automaton is a non deterministic finite automaton).

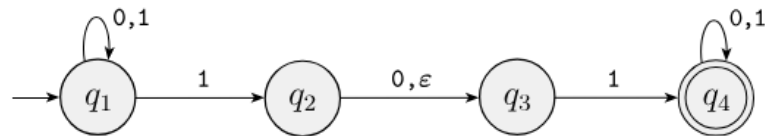


Figure 1.1: Nondeterministic finite automaton

Clearly a deterministic finite automata has only one arrow for each symbol in our alphabet but a non-deterministic can have **zero, one or more**. In addition to symbols of our alphabet we can also have arrows with the label ε which mean a lack